Звіт з програмного проєкту №2

Дослідження швидкодії бібліотечних алгоритмів C++ (`std::execution`) та власного паралельного алгоритму. **Варіант 6** — скалярний добуток.

Виконавець: Корпачов Артем Андрійович, група К-27, факультет комп'ютерних наук та кібернетики КНУ імені Тараса Шевченка (виконується одноосібно). Дисципліна: «Програмування мовою C++. Додаткові розділи».

Стенд: Intel Core i5-9400F (6 ядер / 6 потоків, 2,9 ГГц, турбо 4,1 ГГц, 9 МБ кешу), 15,9 ГБ ОЗУ, Windows 10 (build 19045); паспортна межа пам'яті процесора — 41,6 ГБ/с (DDR4-2666, 2 канали). **Компілятор:** MSVC 19.51.36260, C++23, режим Release (`/O2` та `/Od`). `hardware_concurrency() = 6`.

1. Умова та методика

Досліджується скалярний добуток двох послідовностей `double`: (1) `std::inner_product` без політики; (2) `std::transform_reduce` з політиками `seq`, `par`, `par_unseq`; (3) власний алгоритм `parallel_dot_product`: дані ділиться на K майже рівних частин (розміри відрізняються не більше ніж на 1), кожну частину в окремому потоці обробляє послідовний `std::inner_product`, часткові суми зводить послідовний `std::reduce`. Додатково порівняно дві збірки Release: `/O2` (максимальна оптимізація) і `/Od` (без оптимізації). **Дані:** `std::mt19937` (seed = 42), рівномірний розподіл на [0, 1); розміри 10^5, 10^6, $5{\cdot}10^6$, 10^7, $2{\cdot}10^7$ пар. Послідовності збережено у текстових файлах `data/sequence_N.txt` (перший рядок — N, далі N рядків «a_i b_i»); для всіх експериментів і обох збірок використано **одні й ті самі файли**. Кожен час — **медіана 7 запусків** (експеримент 3 — 15 запусків). **Коректність:** усі алгоритми дали однаковий результат (25039,561912 при N = 10^5), найбільша відносна різниця $1{,}0{\cdot}10^{-14}$.

2. Результати

Таблиця 1. Експерименти 1–2 (збірка /O2): час, мс

N (пар)	inner_product (без політики)	transform_reduce seq	transform_reduce par	transform_reduce par_unseq	par: ефективна пам'ять, ГБ/с
100 000	0,1043	0,0302	0,0382	0,0301	41,9*
1 000 000	1,1507	0,5394	0,3377	0,3506	47,4*
5 000 000	5,8586	3,2543	2,2606	2,2133	35,4
10 000 000	11,5764	6,4846	4,4610	4,4643	35,9
20 000 000	23,4130	13,6739	9,4189	9,3726	34,0

* При N ≤ 10^6 дані повністю чи частково містяться в кеші процесора (9 МБ), тому значення не відображає пропускну здатність ОЗП і може перевищувати її паспортну межу.

Таблиця 2. Експеримент 3: власний алгоритм, N = 20 000 000, час, мс (виділено K = 6, найкраще)

K	1	2	3	4	5	6	7	8	9	10	12	16	24	32	64	128	256	512	1024
/O2	23,43	12,37	10,60	10,13	9,68	**9,40**	11,33	10,39	9,87	9,70	9,47	9,68	9,76	9,86	10,21	11,02	12,08	18,05	35,46
/O2, повтор	23,43	12,47	10,62	10,19	9,82	**9,56**	11,47	10,32	9,96	9,84	9,69	9,82	9,79	9,97	10,22	10,91	12,19	17,87	35,21
/Od	76,64	38,53	26,02	19,93	16,98	**14,98**	22,77	20,06	17,99	16,57	15,05	15,60	16,76	17,34	16,99	17,99	22,01	30,12	45,86

Найкраще K = 6 (9,40 мс); `hardware_concurrency() = 6`; відношення $\mathbf{K_{найкр}}$ **/ hardware_concurrency = 1,00** (у повторному прогоні /O2 та в /Od теж K = 6).

Таблиця 3. Вплив рівня оптимізації (Release, N = 20 000 000)

Алгоритм (N = 20 000 000)	/Od, мс	/O2, мс	Прискорення від /O2
inner_product	76,38	23,41	×3,26
transform_reduce, seq	50,85	13,67	×3,72
transform_reduce, par	12,49	9,42	×1,33
transform_reduce, par_unseq	12,51	9,37	×1,33
parallel_dot_product, K = 6	14,98	9,40	×1,59

3. Відповіді та висновки

1. **inner_product повільніший за transform_reduce навіть без паралелізму:** 23,41 проти 13,67 мс при N = $2{\cdot}10^7$ (у 1,71 раза), а коли дані в кеші (N = 10^5) — у 3,5 раза. `inner_product` витрачає ≈1,04 нс на елемент, тобто близько 4 тактів за турбо-частоти 4,1 ГГц, що відповідає затримці залежного додавання: сума накопичується строго по порядку. `transform_reduce` витрачає ≈0,30 нс (≈1,2 такту) — менше за затримку одного додавання, тож, імовірно, він веде кілька часткових сум (порядок додавання для нього не фіксований). Це гіпотеза, узгоджена з вимірами; дизасемблювання не виконувалось.
2. **Політики виконання.** `par` швидший за `seq` у 1,44–1,45 раза при N ≥ $5{\cdot}10^6$ (1,60 при 10^6), але при N = 10^5 повільніший (0,0382 проти 0,0302 мс): вартість запуску потоків перевищує виграш. `par_unseq` не дає виграшу над `par` (9,37 і 9,42 мс). Причина — пропускна здатність пам'яті: при N = $2{\cdot}10^7$ алгоритм читає 320 МБ, а `par` досягає ≈34 ГБ/с, тобто 82% паспортної межі 41,6 ГБ/с. Тому шість ядер дають лише ×1,45, а додаткова векторизація не дає нічого.

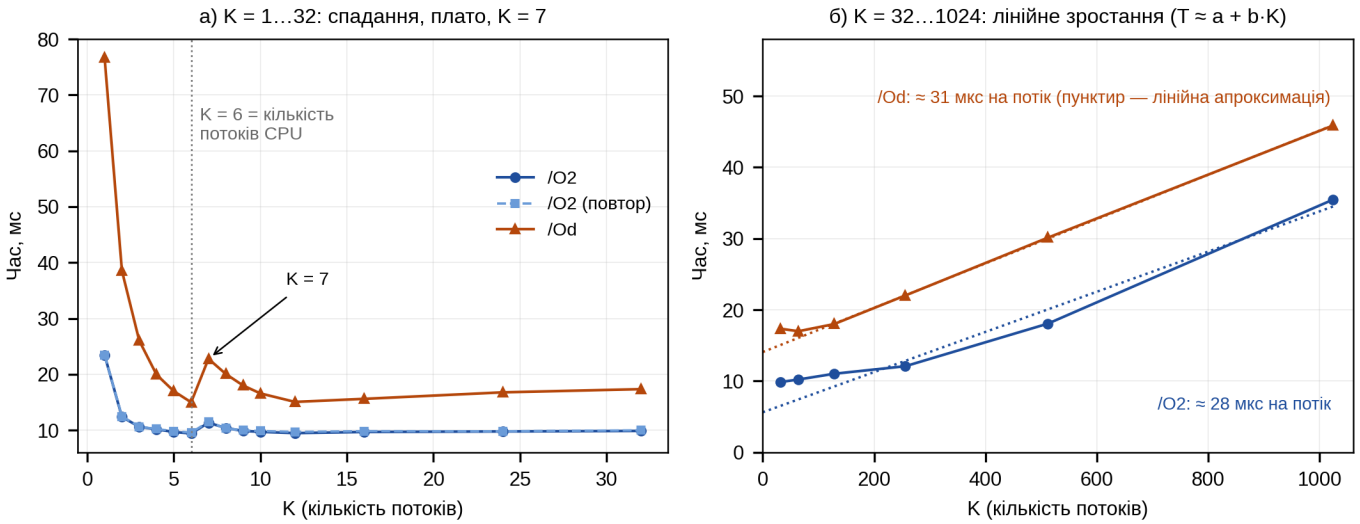


Рис. 1. Час власного алгоритму залежно від K: а) малі K; б) великі K з лінійною апроксимацією за K = 128…1024.

3. **Власний алгоритм: оптимальне K = 6 = hardware_concurrency (відношення 1,00)** у всіх трьох прогонах; час 9,40 мс майже збігається з `transform_reduce(par)` (9,42 мс). **Закон зміни часу з K** має три ділянки:

   (а) **K ≤ 6 — час спадає.** У /Od майже лінійно (прискорення відносно K = 1 для K = 2…6: ×2,0, ×2,9, ×3,8, ×4,5, ×5,1), у /O2 швидко насичується (×1,9, ×2,2, ×2,3, ×2,4, ×2,5). Один потік `inner_product` читає ≈13,7 ГБ/с, тож 3 потоки (≈41 ГБ/с) уже перевищують досяжні ≈34 ГБ/с — пам'ять стає вузьким місцем.

   (б) **K = 5…32 — плато.** У /O2 час у межах 5% від найкращого (K = 5, 6, 9, 10, 12, 16, 24, 32 в обох прогонах), виняток — K = 7 і 8. При K = 7 час на 20% більший (на 52% у /Od), і це відтворюється в усіх прогонах. Імовірна причина: 7 рівних частин на 6 ядрах — одне ядро має опрацювати дві частини; при K, кратному 6 (12, 24), навантаження рівномірне.

   (в) **K ≥ 64 — час зростає приблизно лінійно**, T(K) ≈ a + b·K, де b ≈ 28 мкс на потік у /O2 (апроксимація за K = 128…1024, $R^2 = 0{,}98$) і ≈ 31 мкс у /Od ($R^2 = 0{,}9999$). Це вартість створення та приєднання потоку, яка не залежить від розміру частини; при K = 1024 час у 3,8 раза більший за найкращий.

4. **Рівень оптимізації.** Без оптимізації послідовні алгоритми повільніші у 3,3–3,7 раза (вони обмежені обчисленнями), а паралельні — лише у 1,3–1,6 раза: вони впираються в пропускну здатність пам'яті, яку оптимізація не збільшує. Тому виграш `par` над `seq` у /Od становить ×4,1, а в /O2 — лише ×1,45: паралелізм частково маскує неоптимізований код, але оптимізація компілятора все одно істотна.

5. **Практичний висновок і надійність.** Брати K = hardware_concurrency або кратне йому. Повторний прогін /O2 дав той самий оптимум K = 6 і ту саму множину K у межах 5% від найкращого; різниця між прогонами для K = 6 — 1,7%. Джерела чисел: `results_O2.txt`, `results_O2_repeat.txt`, `results_Od.txt`.